PROGRAMACIÓN AVANZADA · UTU · BACHILLERATO TECNOLÓGICO

Actividades de práctica — Mini-CRUD

Tres actividades progresivas sobre el proyecto MiniCRUD con alta, edición y eliminación: ampliar la entidad, validar los datos en el servidor al agregar y al editar, y construir un CRUD completo para una entidad propia.

Prof. Elizabeth Izquierdo

TEMA 2 — MVC | PROYECTO BASE: MINI-CRUD1 | REFERENCIA: MINI-CRUD2 | DIFICULTAD CRECIENTE

// CONTENIDO

Actividad 01	**Ampliar la entidad Persona** (teléfono y dirección)	BÁSICO
Actividad 02	**Validar al agregar y al editar**	INTERMEDIO
Actividad 03	**Nuevo CRUD** para una entidad a elección	AVANZADO

CÓMO SE ENTREGA

Cada actividad se entrega siguiendo su guía de entrega: `guia-entrega-minicrud_2.html` del repositorio **MiniCRUD-Practicas**. Allí están las capturas que hay que adjuntar y el nombre exacto del archivo .zip.

ACTIVIDAD 01 · NIVEL BÁSICO · PROYECTO BASE: MINI-CRUD1

01 Ampliar la entidad Persona

BÁSICO

MINI-CRUD1 | 5 ARCHIVOS | 6 TAREAS

OBJETIVO

El proyecto base (`mini-crud1`) ya tiene las operaciones de agregar, editar y eliminar funcionando para **código** y **nombre**. La tarea es agregar los atributos **teléfono** y **dirección** y hacer que aparezcan en el formulario de alta, en la tabla de listado y en el formulario de edición.

`PersonaVO: cod, nombre → cod, nombre, telefono, direccion`

// PREGUNTAS ORIENTADORAS — RESPONDER ANTES DE TOCAR CÓDIGO

1. ¿En qué archivo Java se declara la estructura de la entidad Persona? ¿Qué hay que agregar en ese archivo para incorporar dos atributos nuevos: el atributo en sí, el constructor, los getters/setters y el `toString()`?
2. ¿Qué diferencia hay entre el formulario de alta y el formulario de edición en cuanto a cómo manejan los datos de una Persona? ¿Por qué hay que modificar los dos?
3. ¿Cómo llega el valor de un campo de texto desde el formulario hasta el Servlet? ¿Qué método de `HttpServletRequest` se usa para leerlo?
4. En la vista de listado (`persona-lista.jsp`), ¿qué sintaxis de Expression Language (EL) se usa para mostrar el valor de un atributo? ¿Qué condición tiene que cumplir el nombre del getter en `PersonaVO` para que EL lo encuentre?
5. En el formulario de edición, ¿cómo se precargan los valores actuales en los campos del formulario? ¿Qué expresión EL se escribe en el atributo `value` del input?
6. ¿Cuántos archivos JSP hay que modificar en esta actividad? Enumerar cada uno y describir en una línea qué cambia en cada uno.

// ARCHIVOS INVOLUCRADOS

ARCHIVO	CAPA	QUÉ CAMBIA
`PersonaVO.java`	Modelo · VO	Agregar `private String telefono` y `private String direccion`. Actualizar constructor, getters/setters y `toString()`.
`PersonaControladorServlet.java`	Controlador	En `agregar()` y en `editar()`: leer `getParameter("telefono")` y `getParameter("direccion")` y pasarlos al constructor de `PersonaVO`.
`persona-form.jsp`	Vista · Alta	Agregar `<label>` e `<input>` para teléfono y dirección.
`persona-lista.jsp`	Vista · Listado	Agregar las columnas Teléfono y Dirección en el `<thead>` y en cada fila de la tabla.
`persona-editar.jsp`	Vista · Edición	Agregar los dos campos con los valores precargados usando EL.
`PersonaDAO.java`	Modelo · DAO	Sin cambios: el DAO guarda objetos `PersonaVO` completos, sin importar cuántos atributos tengan.

// TAREAS

1. **PersonaVO.java:** declarar `private String telefono` y `private String direccion`. Actualizar el constructor para que reciba 4 parámetros y agregar getter y setter para cada atributo nuevo. Actualizar `toString()`.

2. **Servlet — agregar():** leer `request.getParameter("telefono")` y `request.getParameter("direccion")` y pasarlos al constructor de `PersonaVO`.

3. **Servlet — editar():** leer los mismos dos parámetros y construir el `PersonaVO` con los 4 datos antes de llamar a `dao.actualizar(...)`.

4. **persona-form.jsp:** agregar un `<input type="tel">` para el teléfono y un `<input type="text">` para la dirección, cada uno con su `<label>`. El atributo `name` debe coincidir exactamente con lo que lee el Servlet.

5. **persona-lista.jsp:** agregar `<th>Teléfono</th>` y `<th>Dirección</th>` en el encabezado y las celdas `${p.telefono}` y `${p.direccion}` dentro del `<c:forEach>`. Actualizar el `colspan` de la fila "No hay personas registradas" de 3 a 5.

6. **persona-editar.jsp:** agregar los dos campos precargados con `value="${requestScope.persona.telefono}"` y `value="${requestScope.persona.direccion}"`.

// REFERENCIAS DE CÓDIGO

PersonaVO.java — atributos y constructor nuevos

```
private int cod;
private String nombre;
private String telefono;     // nuevo
private String direccion;    // nuevo

// Constructor actualizado (4 parámetros)
public PersonaVO(int cod, String nombre, String telefono, String direccion) {
    this.cod = cod;
    this.nombre = nombre;
    this.telefono = telefono;
    this.direccion = direccion;
}

public String getTelefono() {
    return telefono;
}

public void setTelefono(String telefono) {
    this.telefono = telefono;
}
// getDireccion() y setDireccion(...) siguen el mismo patrón
```

PersonaControladorServlet.java — agregar() y editar()

```
private void agregar(HttpServletRequest request, HttpServletResponse response)
        throws IOException {
    int codigo = Integer.parseInt(request.getParameter("codigo"));
    String nombre = request.getParameter("nombre");
    String telefono = request.getParameter("telefono");     // nuevo
    String direccion = request.getParameter("direccion");   // nuevo
    dao.agregar(new PersonaVO(codigo, nombre.trim(), telefono, direccion));
    response.sendRedirect(request.getContextPath() + "/persona");
}

// editar(): los mismos dos parámetros, y luego
//     dao.actualizar(new PersonaVO(codigo, nombre.trim(), telefono, direccion));
```

persona-form.jsp — dentro del <form>

```
<label>Teléfono</label>
<input type="tel" name="telefono">

<label>Dirección</label>
<input type="text" name="direccion">
```

persona-editar.jsp — campos precargados

```
<label>Teléfono</label>
<input type="tel" name="telefono"
       value="${requestScope.persona.telefono}">

<label>Dirección</label>
<input type="text" name="direccion"
       value="${requestScope.persona.direccion}">
```

persona-lista.jsp — encabezado y fila

```
<th>Código</th>
<th>Nombre</th>
<th>Teléfono</th>
<th>Dirección</th>
<th>Acciones</th>

<!-- dentro de <c:forEach var="p" ...> -->
<td>${p.codigo}</td>
<td>${p.nombre}</td>
<td>${p.telefono}</td>
<td>${p.direccion}</td>
```

// PARA VERIFICAR

- ✓ El campo nuevo aparece en el formulario de alta, en la tabla del listado y precargado en el formulario de edición. Si falta en alguno, revisar el archivo correspondiente.
- ✓ Al editar y guardar, los valores nuevos de teléfono y dirección se ven en el listado.

> **ERRORES FRECUENTES**
>
> Falta un getter en `PersonaVO`, el atributo `name` del input no coincide con el parámetro que lee el Servlet, o el constructor de `PersonaVO` no se actualizó con los campos nuevos.

02

ACTIVIDAD 02 · NIVEL INTERMEDIO · PROYECTO BASE: MINI-CRUD1 O LA ACTIVIDAD 01

Validar al agregar y al editar

INTERMEDIO

MINI-CRUD1 | 3 ARCHIVOS | SIN CAMBIOS EN PERSONAVO

OBJETIVO

Agregar validación del lado del servidor en dos operaciones.
Al agregar: incorporar la validación de `agregar()` explicada en la wiki (página **Validación de campos obligatorios**): código y nombre obligatorios, código entero mayor que 0 y no repetido.
Al editar: aplicar el mismo criterio en `editar()`, que no está resuelto en la wiki. Si el nombre llega vacío, o si el código llega vacío, no es un número o no corresponde a una persona existente, no guardar y volver al formulario de edición con un mensaje de error, conservando los datos ya escritos. Si los datos son válidos, actualizar y redirigir al listado (patrón PRG).

FORWARD VS. REDIRECT

Forward (`getRequestDispatcher(...).forward()`): sigue en la misma petición; los atributos del request y los parámetros del formulario siguen disponibles en el JSP. Se usa cuando hay un error y hay que volver al formulario.
Redirect (`sendRedirect()`): el navegador hace una petición nueva y se pierde todo lo del request. Se usa cuando la operación salió bien (PRG).

// PREGUNTAS ORIENTADORAS — RESPONDER ANTES DE TOCAR CÓDIGO

1. ¿Qué significa "validación del lado del servidor"? ¿Por qué no alcanza con usar el atributo `required` en el HTML del formulario?
2. En `agregar()` el código **no** debe existir en el DAO; en `editar()` el código **sí** debe existir. ¿Por qué cambia la condición? ¿Qué método del DAO sirve para comprobarlo en los dos casos?
3. En `persona-editar.jsp` el campo código es `readonly`. ¿Alcanza con eso para no validarlo en el Servlet? ¿De qué otra forma podría llegar un código modificado o vacío?
4. Si el Servlet hace `forward` a `persona-editar.jsp` con un mensaje de error, ¿cómo llega ese mensaje hasta el JSP? ¿Qué se usa en el Servlet para enviarlo y en el JSP para mostrarlo?
5. `persona-editar.jsp` precarga los campos con los datos de la persona que envía `cargarEditar()`. Cuando vuelve con un error, ¿de dónde tiene que leer los valores para mostrar lo que el usuario acababa de escribir?
6. Describir el flujo completo cuando un usuario edita una persona y deja el nombre vacío: qué recibe el Servlet, qué verifica, qué hace si hay error y qué hace si los datos son válidos.

// ARCHIVOS INVOLUCRADOS

ARCHIVO	CAPA	QUÉ CAMBIA
`PersonaControladorServlet.java`	Controlador	En `agregar()`: incorporar la validación de la wiki. En `editar()`: validar el nombre y el código (no vacío, número y persona existente). Si hay error, guardar el mensaje con `setAttribute("error", ...)` y hacer `forward` al formulario que corresponda. Si los datos son válidos, guardar o actualizar y hacer `sendRedirect` al listado.
`persona-form.jsp`	Vista · Alta	Mostrar el mensaje de error cuando existe y conservar el valor que el usuario ya escribió en cada campo.
`persona-editar.jsp`	Vista · Edición	Mostrar el mensaje de error cuando existe y conservar lo que el usuario escribió al volver con un error.

// TAREAS

1. **Servlet — agregar():** reemplazar el método por la versión de la wiki (Validación de campos obligatorios). Agregar `ServletException` al `throws`, porque el método ahora hace `forward`.
2. **persona-form.jsp:** agregar la directiva `taglib` de JSTL, el bloque `<c:if>` que muestra `${requestScope.error}` y precargar los campos con `${param.codigo}` y `${param.nombre}`.
3. **Servlet — editar():** leer el código como `String` y validar, en este orden: que el código no esté vacío, que sea un número entero y que `dao.obtenerXCodigo(codigo)` no devuelva `null`. Validar también que el nombre no esté vacío.
4. **Servlet — editar() con error:** guardar el mensaje con `request.setAttribute("error", ...)`, hacer `forward` a `/WEB-INF/vista/persona-editar.jsp` y terminar el método con `return`. Si los datos son válidos: `dao.actualizar(...)` y `sendRedirect` al listado.
5. **persona-editar.jsp:** mostrar el error con el mismo `<c:if>` del formulario de alta. Al volver con un error, los campos tienen que mostrar lo que el usuario escribió (los parámetros del request) y no los datos guardados de la persona.

// REFERENCIAS DE CÓDIGO

PersonaControladorServlet.java — estructura sugerida para editar()

```java
private void editar(HttpServletRequest request, HttpServletResponse response)
        throws IOException, ServletException {
    String codigoStr = request.getParameter("codigo");
    String nombre = request.getParameter("nombre");
    String error = null;
    int codigo = 0;

    // 1. Código: no vacío, número entero y persona existente
    // 2. Nombre: no vacío

    if (error != null) {
        request.setAttribute("error", error);
        request.getRequestDispatcher("/WEB-INF/vista/persona-editar.jsp")
               .forward(request, response);
        return;
    }

    dao.actualizar(new PersonaVO(codigo, nombre.trim()));
    response.sendRedirect(request.getContextPath() + "/persona");
}
```

persona-editar.jsp — mostrar el error

```jsp
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<c:if test="${requestScope.error != null}">
    <div style="color: red; border: 1px solid red; padding: 10px;
                margin-bottom: 15px; background-color: #ffe8e8;">
        <strong>Error:</strong> ${requestScope.error}
    </div>
</c:if>
```

// PARA VERIFICAR

- ✓ **Al agregar:** probar con el código vacío y luego con el nombre vacío; en los dos casos debe mostrar el error y conservar los demás valores.
- ✓ **Al editar:** abrir una persona con el botón Editar, borrar el nombre y guardar; debe volver al formulario de edición con el error y el código conservado.
- ✓ Editar con datos válidos: debe actualizar y redirigir al listado.
- ✓ Para probar sin la validación del navegador, agregar `novalidate` al `<form>`.

ERRORES FRECUENTES

Si al enviar con error la página queda en blanco o redirige igual al listado, revisar que se usa `forward` (no `sendRedirect`) cuando hay error y que hay un `return` después del forward. Si el formulario de edición vuelve vacío, revisar de dónde lee los valores el JSP cuando llega desde el error.

PARA INVESTIGAR

- Mostrar **todos los errores juntos** en lugar de cortar en el primero: guardar varios mensajes en una lista en el Servlet y recorrerla en el JSP.
- Validar el **formato del nombre**: que tenga al menos 3 caracteres y solo letras y espacios. Investigar qué métodos de `String` y de `Character` permiten comprobarlo.

ACTIVIDAD 03 · NIVEL AVANZADO · REFERENCIA: MINI-CRUD2

03 Nuevo CRUD: entidad a elección

AVANZADO

MINI-CRUD2 (REFERENCIA) | 6 ARCHIVOS NUEVOS | 7 TAREAS

OBJETIVO

Construir un CRUD completo para una entidad a elección: Producto, Libro, Película, Videojuego, Estudiante u otra. Usar `mini-crud2` como referencia: copiar la estructura de cada archivo, cambiar el nombre de la entidad y adaptar los atributos. `mini-crud2` ya resuelve las cuatro operaciones, el patrón PRG y la validación de unicidad del código.

// ELEGIR UNA ENTIDAD

Producto
código (int) · nombre (String) · precio (double) · descripción (String)

Libro
código (int) · título (String) · autor (String) · año (int)

Película
código (int) · título (String) · director (String) · año (int)

Videojuego
código (int) · título (String) · género (String) · plataforma (String)

Estudiante
código (int) · nombre (String) · apellido (String) · grupo (String)

Otra entidad
atributos a acordar con la docente

// PREGUNTAS ORIENTADORAS — RESPONDER ANTES DE TOCAR CÓDIGO

1. ¿Qué archivos componen el patrón MVC en este proyecto? Describir en una oración la responsabilidad de cada capa (Modelo, Vista, Controlador).
2. ¿Para qué sirve la anotación `@WebServlet`? ¿Qué pasa si la URL configurada no coincide exactamente con la URL que se intenta acceder en el navegador?
3. ¿Cómo decide el Servlet qué operación ejecutar (listar, agregar, editar, eliminar)? ¿Qué parámetro se usa y cómo se lee?
4. ¿Qué ventaja tiene separar la lógica de datos en un DAO en lugar de escribirla directamente en el Servlet?
5. En el formulario de edición, ¿por qué el campo del identificador (código o ID) debe ser de solo lectura (`readonly`)? ¿Qué pasaría si el usuario pudiera cambiarlo?
6. Al eliminar un registro, ¿el Servlet debe hacer `forward` o `sendRedirect`? ¿Por qué? ¿Qué pasa si el usuario recarga la página después de eliminar?

// ARCHIVOS A CREAR (EJEMPLO CON PRODUCTO)

ARCHIVO	CAPA	QUÉ CAMBIA
ProductoVO.java	Modelo · VO	Atributos privados, constructor completo, getters/setters y `toString()`.
ProductoDAO.java	Modelo · DAO	Un `ArrayList` en memoria y los mismos métodos que `PersonaDAO` de `mini-crud2`: `getLista()`, `agregar()`, `obtenerXCodigo()`, `actualizar()` y `eliminar()`.
ProductoControladorServlet.java	Controlador	`@WebServlet("/producto")`, leer el parámetro `action` y delegar en el DAO. Redirigir o hacer forward según el resultado.
producto-form.jsp	Vista · Alta	Un campo por atributo, POST al Servlet y mensaje de error si el código ya existe.
producto-lista.jsp	Vista · Listado	Tabla con todos los atributos y los botones Editar y Eliminar en cada fila.
producto-editar.jsp	Vista · Edición	Campos precargados con EL; el código, `readonly`.

// TAREAS

1. Elegir la entidad y anotar sus atributos con el tipo de dato Java de cada uno. Si se elige "Otra entidad", acordar los atributos con la docente antes de seguir.
2. **XxxVO.java:** crear la clase con los atributos, el constructor con todos los parámetros, el constructor vacío, getters, setters y `toString()`.
3. **XxxDAO.java:** crear el DAO copiando `PersonaDAO` de `mini-crud2` y adaptándolo a la entidad, con los mismos nombres de métodos.
4. **XxxControladorServlet.java:** mapear las acciones en `doGet()` (agregar, cargarEditar, listar) y en `doPost()` (agregar, editar, eliminar). En `agregar()`, comprobar la unicidad con `dao.obtenerXCodigo(codigo)`: si el código existe, `forward` con error; si no, guardar y `sendRedirect`.
5. **xxx-form.jsp:** formulario de alta con un campo por atributo, el bloque de error con JSTL y los campos precargados al volver con un error.
6. **xxx-lista.jsp:** tabla con una columna por atributo y la columna de acciones, recorriendo la lista con `<c:forEach>`. Eliminar siempre por POST.
7. **xxx-editar.jsp:** formulario de edición con todos los campos precargados con EL y el código en `readonly`.

// PATRÓN CLAVE: VALIDACIÓN DE UNICIDAD DEL CÓDIGO (MINI-CRUD2)

PersonaControladorServlet.java (mini-crud2) — agregar()

```
private void agregar(HttpServletRequest request, HttpServletResponse response)
        throws IOException, ServletException {
    int codigo = Integer.parseInt(request.getParameter("codigo"));
    String nombre = request.getParameter("nombre");
    PersonaVO personaParaFormulario = new PersonaVO(codigo, nombre.trim());

    // Verificación de unicidad usando el DAO
    if (dao.obtenerXCodigo(codigo) != null) {
        request.setAttribute("error",
                "Error: El código " + codigo + " ya está registrado. Ingrese uno diferente.");
        request.setAttribute("persona", personaParaFormulario);   // mantener datos
        request.getRequestDispatcher("/WEB-INF/vista/persona-form.jsp")
               .forward(request, response);
        return;
    }

    dao.agregar(personaParaFormulario);
    response.sendRedirect(request.getContextPath() + "/persona");   // PRG
}
```

// PARA VERIFICAR

- ✓ Se puede dar de alta un registro con todos los campos completos.
- ✓ Si se intenta dar de alta con un código ya existente, el sistema muestra error y no guarda.
- ✓ La tabla del listado muestra todos los registros con sus atributos.
- ✓ Se puede editar un registro y los cambios se ven en el listado.
- ✓ Se puede eliminar un registro y desaparece del listado.
- ✓ En alta y edición se aplica el patrón PRG: el éxito redirige; el error hace forward.

> **ERRORES FRECUENTES**
>
> Si el navegador muestra 404, verificar que el mapeo `@WebServlet` coincide exactamente con la URL a la que se intenta acceder. Si hay error 500, revisar la consola de Tomcat para ver el mensaje de la excepción.